

Review

Artificial grammar learning meets formal language theory: an overview

W. Tecumseh Fitch^{1,*} and Angela D. Friederici²

¹*Department of Cognitive Biology, University of Vienna, Althanstrasse 14, Vienna 1090, Austria*

²*Max Planck Institute for Human Cognitive and Brain Sciences, Stephanstrasse 1a, 04103 Leipzig, Germany*

Formal language theory (FLT), part of the broader mathematical theory of computation, provides a systematic terminology and set of conventions for describing rules and the structures they generate, along with a rich body of discoveries and theorems concerning generative rule systems. Despite its name, FLT is not limited to human language, but is equally applicable to computer programs, music, visual patterns, animal vocalizations, RNA structure and even dance. In the last decade, this theory has been profitably used to frame hypotheses and to design brain imaging and animal-learning experiments, mostly using the ‘artificial grammar-learning’ paradigm. We offer a brief, non-technical introduction to FLT and then a more detailed analysis of empirical research based on this theory. We suggest that progress has been hampered by a pervasive conflation of distinct issues, including hierarchy, dependency, complexity and recursion. We offer clarifications of several relevant hypotheses and the experimental designs necessary to test them. We finally review the recent brain imaging literature, using formal languages, identifying areas of convergence and outstanding debates. We conclude that FLT has much to offer scientists who are interested in rigorous empirical investigations of human cognition from a neuroscientific and comparative perspective.

Keywords: artificial grammar learning; formal language theory; comparative neuroscience; neurolinguistics

1. INTRODUCTION

Formal language theory (FLT) has its roots in mathematics [1,2] but was established in its modern form by Noam Chomsky in an attempt to systematically investigate the computational basis of human language [3,4]. Since these beginnings, the theory has been continually expanded to cover other scientific domains. The most prominent new application was in computer science, where the study of FLT is now a core part of the standard curriculum, providing the theoretical foundation for fundamental issues such as programming language structure and compiler design [5]. Psychologists have used FLT to explore learning and pattern-processing abilities in humans and other species [6–10], while in neuroscience the theory has been used in neuroimaging experiments to better understand the neural computations of hierarchy and sequence [11,12]. Finally, in biology, FLT has been used to analyse diverse topics such as the structure of RNA molecules [13,14] and the sequential structure of chickadee song [15]. FLT has thus grown far beyond its original roots in language, to become a key component

of the theory of computation, applicable to virtually any rule-governed system, in any domain.

In this paper, we review recent progress in applying FLT to empirical research in animal cognition and neuroscience, as well as highlighting some pitfalls that can accompany attempts to merge theory and practice. We start with a non-technical overview of FLT, intended to give an intuitive understanding of the theory and its significance and to provide a gentle preparatory overview for the more rigorous paper by Jaeger & Rogers [16]. We then provide a more detailed analysis of the difficulties involved in translating this body of theory into an empirical research programme. We start with the difficulties caused by the use of infinity as a tool for proofs in mathematics, which leaves such proofs technically irrelevant in the real world of finite brains and finite time. We provide a detailed analysis of one particular rule system, the so-called ‘AⁿBⁿ’ grammar, which has been employed in many recent studies, both in neuroscience and in animal cognition. We suggest that this grammar is appropriate for answering certain interesting questions, but has sometimes been over-extended to address issues for which it is poorly suited, for which we suggest alternative, more appropriate grammars. In the process, we highlight the need to clearly distinguish among a number of separate issues, which—although related—should not be conflated.

* Author for correspondence (tecumseh.fitch@univie.ac.at).

One contribution of 13 to a Theme Issue ‘Pattern perception and computational complexity’.

These include notions such as hierarchical structure versus centre-embedding, context-freeness versus long-distance dependency and formal complexity versus recursion. FLT provides the theoretical concepts and terminology to clearly distinguish among all of these terms, and we argue that it should be used to do so more rigorously in the future. We then provide a detailed look at some of the recent brain imaging literature using FLT, highlighting the areas of nascent agreement along with outstanding open questions. We conclude by pointing out some areas within FLT that remain little explored, but might provide fertile ground for future research.

2. FORMAL LANGUAGE THEORY AND THE THEORY OF COMPUTATION

We have an intuitive sense that some cognitive computations are more difficult than others. For most people, it is harder to play chess or solve equations than to buy groceries or drive a car. For most of us, it is more difficult to parse sentences in a non-native language (regardless of our level of proficiency) than in our native language. However, a central finding of computer science is that our intuitions about complexity do not necessarily apply to computer programs. In fact, it has proved relatively easy to create machines that can play chess at a high level, but so far impossible to create adequate car-driving systems. Because of this, an important component of modern computer science is a framework for quantifying the ‘difficulty’ or ‘complexity’ of a computational problem or algorithm in terms that are explicit and unambiguous. Starting with the work of the brilliant mathematician Alan Turing, and combined with further insights owing to Gödel, Church, Post, Kleene, Chomsky and many others, FLT has grown today into one key pillar of the theory of computation (and thus compiler design and many other aspects of computer science). The other main pillars are the theory of computability (what problems can or cannot be solved) and the theory of problem complexity (how the difficulty of problems scales with their size) [5,17].

The theory of computation provides the practical basis for software tools we use everyday, which thus provide useful illustrations of the core concepts of FLT. We favour such everyday examples from computers, rather than mathematical formalisms, because we expect that most of our readers will have some experience with the former but not necessarily with the latter. More detailed and mathematical treatments are easy to find [5,17–20], and a paper in this issue provides a particularly accessible formal introduction designed for experimentalists [16].

(a) *Regular expressions*

We start our survey with a simple, well-defined computational system, termed a finite-state machine, which is equivalent to another simple construct called a ‘regular expression’. Search functions, such as the *dir* command in DOS, or the *ls* function in UNIX, provide everyday examples. Such functions use a syntax that allows us to search for the arbitrary

target pattern in a large database of words and/or numbers. Given a set of file names:

```
> filenames = {a.wav, b.doc, c.bmp,
MySong.doc, MySong.wav}
```

running the function:

```
> ls *.wav filenames
```

(or the equivalent with *dir* in DOS) on this set will return the subset

```
> {a.wav, MySong.wav}.
```

The search string ‘*.wav’ says, in effect, ‘give me all the strings that end with “.wav”’. The * character tells the *ls* or *dir* command that the string(s) can start with any characters in the alphabet, we do not care which or how many. This search string is one simple example of a general framework called ‘regular expressions’, which provide a very powerful basis for computer-based search that underlies searching, replacing and other functions in many computer programs. This ability to use regular expressions to match patterns was first instantiated in the *grep* function in UNIX/Linux, and has proved so useful that the term ‘grep’ has entered hacker lingo as a verb meaning ‘to search by computer’.

Regular expressions are composed using a few simple but powerful rules and operators, familiar to many computer users. The operator *, as used earlier, means ‘any string of any length’ and by appending it to our search string (e.g. ‘string’), we can find our target pattern even if preceded by anything (*string), followed by anything (string*) or buried in anything (*string*).¹ More specific operators also allow us to specify a single, unspecified character (?), a character from a particular set (e.g. numbers {0–9} versus letters {a–z}), or even a specific number of characters from a certain set. Any time you have some pattern or a set of patterns that can be captured by a regular expression, you can use *grep* to search an arbitrary database for that pattern. You can *grep* for your name or email address or telephone number in the archives of a discussion group, or *grep* for a particular gene sequence in the online human genome database. The search engine Google is an extended version of *grep* that takes the entire web as its database. Regular expressions are at the core of computer search in today’s world.

Given this flexibility and power, we might think that regular expressions are capable of specifying any kind of pattern that we can imagine. Crucially, however, this turns out not to be true. For instance, imagine a simple symmetrical pattern where a particular number of items of type A is followed by the same number of a different type B. Examples of this set include {AB, AAB, AAAB, AAAAB, etc.}, and extend indefinitely (so a string of 1346 ‘A’s followed by 1346 ‘B’s is still a member of the set). This pattern is notated $A^n B^n$ in FLT. It is easily proved that this set cannot be specified by a regular expression (see the textbooks listed earlier for mathematical proofs). We conclude from this fact that there are patterns that we can conceive of, and that we could easily (if laboriously) recognize ourselves, but that cannot be captured by a regular expression. Why does this matter? Because, as demonstrated by the mathematician Stephen Kleene

in 1956 in the theorem that bears his name, regular expressions and the corresponding rule sets termed ‘regular grammars’ are exactly equivalent to one of the most ubiquitous classes of computing devices, which are termed ‘finite-state automata’ [21].

(b) Finite-state automata

FLT relies on abstract models of computational systems termed ‘automata’ (and often, perhaps confusingly, also often called ‘machines’). Two canonical examples of such models are the finite-state automaton (FSA) and the Turing machine. Automata such as these are mathematical abstractions, not real devices designed to be manufactured. For example, the Turing machine includes as part of its mathematical definition a storage tape of infinite length, and thus we could never build a real Turing machine. Many automata, although well-defined in theory, are unbuildable in practice (a fact that has some implications that we will discuss later). Despite this, the abstract notion of a Turing machine is extremely important and powerful in mathematics and FLT: infinity is a powerful tool for mathematical abstraction, but not a real thing that we find in the world.

The simplest class of well-defined automata are called finite-state automata because they have a finite number of operating states or ‘positions’. The FSA starts at a predefined start state, and then jumps between its other states, depending only on its current input and current state. For each of these jumps, it can emit an output symbol as it hops along. Thus, an FSA can be fully defined by its set of states, its input alphabet (the input symbols that it recognizes), an optional output alphabet (which might or might not be different) and a function that tells it which is the next state to go to, given its current state and current input. Any given FSA is capable of ‘recognizing’ a certain set of patterns, and rejecting others. By ‘recognize’ we mean simply that, given this pattern as input, it can generate some particular prespecified output (e.g. ‘OK’). The set of patterns recognizable by an FSA may be infinite. Because of this, and its simplicity, the FSA is a good starting point for further discussions of automata and computational complexity. Critically, as already mentioned, Kleene’s Theorem demonstrates the equivalence, or interchangeability, of FSAs and regular expressions.

One point, overlooked by many, is that FSAs can recognize (or generate) a simple, long-distance dependency of the start-and-end sort (e.g. ab^*a or cd^*c) as the automaton in figure 1c shows (p. 1103). However, other patterns are clearly beyond the capabilities of an FSA, because we already know that certain patterns, such as A^nB^n , cannot be captured by regular expressions: they are beyond the capabilities of our simplest class of automaton. Thus, something with more computational power is clearly needed.

(c) Turing machines

Other automata take an FSA as their starting point, and achieve additional computational power by adding some additional form of memory. The most important and powerful such automaton is the Turing machine, which adds to an FSA (the ‘controller’) a

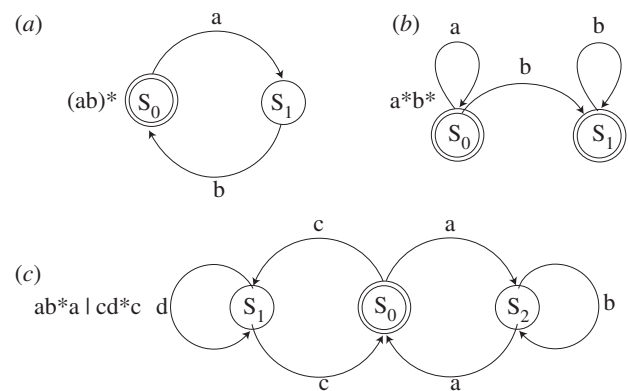


Figure 1. Three examples of simple finite-state automata and their stringsets. Circles represent states, arcs represent transitions between states, with the corresponding symbols, and double circles indicate ‘accept’ states. (a) The $(ab)^*$ or $(ab)^n$: accepts strings of zero or more ‘ab’ bigrams. (b) The a^*b^* : accepts any number of ‘a’s followed by any number of ‘b’s. (c) A long-distance dependency: this automaton illustrates that FSAs can also check for arbitrarily long-distance dependencies. This grammar accepts strings of the form ab^*a , where any number of ‘b’s can intervene between the two ‘dependent’ ‘a’s, (or similarly for cd^*c strings).

storage tape of unbounded length, on which symbols can be written or erased.² Thus, in addition to possessing a large (but finite) set of states that its controller can occupy at any one moment, the Turing machine has by virtue of this tape an additional unlimited form of memory for storing past operations, intermediate results and so on. A Turing machine can easily recognize the A^nB^n language described earlier: it simply stores the number of times A has been repeated (that is, writes successive integers every time its input jumps from A to A) and then compares that with the number of Bs (B to B jumps). Thus, Turing machines are more ‘powerful’ than their FSA component, in the sense that they can recognize patterns and solve problems unsolvable by any FSA. This is not surprising given their additional resources. What is altogether more remarkable is that the Turing machine is capable of computing ANY deterministic function whatsoever: if something is computable, a Turing machine can compute it. Thus, modern computer scientists accept the Turing machine as their very definition of ‘computability’, broadly accepting the ‘Church/Turing thesis’ that a function is computable if and only if it is computable by a Turing machine.

The Turing machine and FSA are well-defined automata that provide useful endpoints for a scale of *computational power*: the FSA provides the lower level (which is powerful and practically useful, but has its limits), while the Turing machine provides the upper limit (it is all-powerful in the sense that, if a function is computable at all, an automaton in this class can compute it). We will use the term ‘computational power’ in this paper in this specific sense, framed by the specific automata discussed in FLT. We do not mean to imply by this that this is the only way to insightfully characterize the power of algorithms, or that ultimately this is the best way to think

about the different aspects of brain function. What this specific sense of computational power gives us is an explicit, formal axis along which any particular algorithm can be placed, which thus provides one useful dimension along which to characterize the rule-governed capacities of a machine, or a human or animal subject. Other potentially useful dimensions will be discussed briefly at the end of this paper.

Given these two endpoints, we might immediately ask two questions:

- Are there other intermediate classes of automata, with powers greater than an FSA but less than a Turing machine?
- Where do human computational powers (or those of other species) fall along this spectrum?

(d) *The Chomsky hierarchy*

In an attempt to answer the second question, the young Noam Chomsky and his colleagues built upon the framework already discussed and provided a positive answer to the first question. Chomsky outlined a set of intermediate formal possibilities, between the extremes of Turing machines and finite-state automata, and arranged them in a theoretical hierarchy that now bears his name. Because both the nature and the importance of this hierarchy are sometimes misunderstood, we will try to make clear here both what the Chomsky hierarchy is, and why it is important. First, let us consider the relationship between a Turing machine and an FSA. Because every Turing machine contains within it an FSA, anything computable (e.g. any pattern that can be recognized) by an FSA is perforce computable by a Turing machine. Thus, the set of FSA-recognizable patterns is a proper subset of those computable by a Turing machine. This is obvious from the way in which these automata are defined.

The Chomsky hierarchy incorporates several intermediate levels of automata, which have in common with the Turing machine an additional memory system but discard the assumption that this memory can be freely accessed. For example, a ‘pushdown automaton’ (PDA) includes an FSA and a pushdown stack (which is a memory that can only return the most recent item placed upon it, like the stack of trays in a cafeteria). Because a stack is more limited than the infinite tape, it is intuitive (and can be shown mathematically) that the PDA is less powerful than a Turing machine, while being more powerful than an FSA. And as before, the set of patterns recognizable by the PDA is a proper subset of those captured by the Turing machine. In fact, a PDA can recognize the A^nB^n language discussed earlier, which is beyond an FSA. Thus, we now have a nested set of patterns, enclosed one within the other like Russian dolls. A second intermediate form of automaton is called a linear-bounded automaton and includes both the FSA and PDA within it. Figure 2 provides an illustration of this nested hierarchy.

The Chomsky hierarchy provides a broad framework for discussing the computational power of automata, universally accepted in this role in theoretical computer

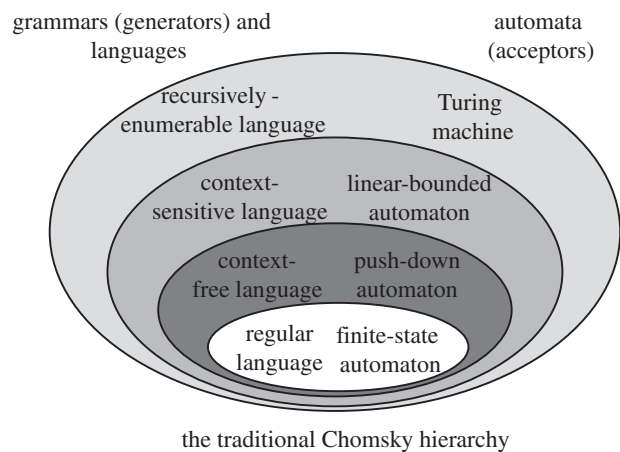


Figure 2. The Chomsky hierarchy for grammars, languages and automata. The Chomsky hierarchy is an inclusion hierarchy that aligns particular forms of grammar, and the languages they generate, with particular classes of automata—abstract computational ‘machines’ that can be constructed to accept such languages. All of the grey circles beyond the innermost white circle represent the *supra-regular* grammars and languages, which require computational power above the level of a finite-state automaton. See Jäger & Rogers [16] for more details.

science, algorithmic theory, FLT and discrete mathematics. It is well-defined, explicit and unambiguous and discussed and defined in any textbook on these topics. The Chomsky hierarchy contains no hidden assumptions about Universal Grammar, the ‘poverty of the stimulus’ argument or other of Chomsky’s various more controversial ideas about language. A computer scientist using the Chomsky hierarchy need not accept other aspects of Chomsky’s thought, any more than a logician using Bertrand Russell’s innovations in mathematical logic need accept Russell’s pacifism or atheism. While this should be an obvious point, we have been surprised by how often the distinction gets blurred. Although there are other ways to arrange automata in hierarchies of ascending power, as well as finer subdivisions of existing hierarchies [16], we focus on the Chomsky hierarchy in this work owing to its interdisciplinary acceptance and understanding. Anyone who studies basic computer science or the theory of algorithms will be familiar with the framework, and this is more than can be said for any other framework we know of. We do not claim that this whole set of automata (including FSAs and Turing machines) is the best or the most insightful way of arranging the different types of neural computations that we ultimately want to understand as psychologists, biologists or neuroscientists [22]. Indeed, as our understanding of neural computation in vertebrates progresses, it seems likely that different hierarchies will arise, and prove to be more useful. Until such progress is made, however, FLT seems to provide the best theoretical starting point, and indeed has no obvious competition.

One might object that some models of neural computation, especially connectionist networks, offer just such an alternative framework. This is not true. In fact, neural networks are automata, like any other well-defined computational system. Indeed,

surprisingly simple recurrent neural networks can be shown to be Turing-equivalent in principle [23] (although programming such networks to do some of the tasks they could perform in theory has been difficult or impossible in practice). Other classes of automata, such as augmented transition networks, also take their place within this scheme. The power of the Turing machine framework is that it includes *any* computational device: the definition is formally equivalent to a vast number of alternative implementations, in a wide variety of forms (indeed many different computational frameworks were initially offered as alternatives to Turing machines, but were later shown to be Turing-equivalent). Thus, the term ‘computation’ is used here, and in computer science in general, in a very broad and inclusive sense to capture any algorithm (information processing system). Neural networks and a vast array of other implementations are part of this classification, not alternatives to it.

(e) *Formal language theory*

With these preliminaries behind us, and the basic aims and principles of computational theory clarified, we can now introduce the terminology and principles of FLT. On the one hand, fortunately, this terminology is simple and unambiguous, and thus quite familiar because it maps the core technical concepts onto everyday terms. On the other hand, this familiarity can be deceptive and misleading. As typical with technical terms, we must beware of unwittingly slipping from interpreting the words in their technical sense to their broader everyday sense.

First, the terminology: an *alphabet* A is a set containing a finite number of indivisible symbols. A *string* (often termed, more confusingly, a *word* or a *sentence*) is a finite sequence of symbols, and a *string-over- A* is a finite sequence of symbols from A . The set of all such strings-over- A is denoted A^* , pronounced ‘ A star’.³ Finally, a *language* over A is any subset of A^* . Put verbally, a ‘language’ in this abstract sense is some set of ‘legal’ strings from our alphabet A . Put concretely, let’s say an alphabet A_1 consists of the digits 0–9. Then any integer number is a member of a language defined over this alphabet (a member of the language denoted by A_1^*), but the number 1.35 is not (because the ‘.’ is not in our alphabet). The string ‘cat’ is, for the same reason, not a member of this language. Similarly, if our alphabet consists of the letters of the Roman alphabet a–z, we could define some more specific languages where ‘cat’ is contained in the language but ‘katze’ is not.

These terms and their definitions seem quite intuitive and easily understood. The danger is that they are so easy to understand that it is easy to forget what they do and do not involve. Most prominently, there is no discussion of meaning within this framework. Although we can easily design a grammar, in the technical sense, that can accept ‘cat’ while rejecting ‘katze’, this system has no understanding that these strings mean ‘feline animal’, or indeed that they could mean anything at all. A grammar responds purely to sequence and has no way of embodying meaning or

of ‘understanding’ the signals that are fed to it. These systems are purely syntactic and have no semantics. Thus, ‘language’ in FLT is missing a substantial component of natural language (the central component of meaning). A ‘language’ in this formal context is a set of strings (a ‘stringset’), and nothing more.

Continuing our exploration of FLT, the simplest way to define a finite language is to simply list all its members. This ‘brute force’ approach is a possibility as long as the language is finite, e.g. for the set containing the words of English (e.g. a dictionary omitting names, loan words or neologisms). It will not, however, be able to deal with the integers, or the sentences of English, because the number of integers or English sentences is unlimited, and the list could never be complete (any candidate finite set can always be ‘trumped’ by adding ‘Mary thinks that’ to a randomly chosen member of the set). If the language in question is infinite, it cannot be listed, and our only hope is to come up with some finite set of rules to generate the language, termed a grammar. A *grammar* is a finite set of rules that specifies some (typically infinite) language. Note that the sentences making up such a set will themselves be finite (in the same way that the set of integers is infinite but each integer is itself finite).⁴

We are finally in a position to restate the Chomsky hierarchy in explicit formal terms. The Chomsky hierarchy incorporates a nested set of automata of increasing power, each of which can generate the strings of some formally defined class of languages. The automata (which have already been discussed) are shown with their corresponding formal language classes, in figure 2. Our old friend, the FSA, is at the centre, with its corresponding language family, the regular languages. Next come the context-free languages, defined by PDAs, which are subsets of context-sensitive languages defined by linear-bounded automata. Finally, in the outer and most powerful ring, we find the languages recognizable by Turing machines. The grammars defining these languages have been given many names, but they were called ‘type 0’ by Chomsky (who dubbed a certain subset of these grammars ‘transformational grammars’ [24]).

A subtle terminological difficulty arises from the nested aspect of the rule systems just described. Because FSAs are contained within the class of PDA, the term ‘pushdown automaton’ *sensu latu* logically includes all FSAs as a subset or special case. However, these terms are often used informally to exclude their subsets, much as the term ‘reptile’ is used by biologists to delineate all those descendants of the ancestral reptile who are not birds or mammals (the correct term ‘non-avian non-mammalian amniotes’ being a bit of a mouthful). Because this type of usage is quite indispensable in the research programme outlined here, we will explicitly define it. Using basic set theory, we can define any of these automata *sensu strictu* in an exclusive manner. Thus, we could use the term ‘PDA *sensu strictu*’ to delineate all PDAs that are not simply FSAs (in logical terms, this is the set of type 2 grammars omitting the set of type 3 grammars). Of particular importance in the present context are the ‘supra-regular grammars *sensu*

strictu', which we define as the class of all automata (that is, all Turing machines) that are not simply FSAs (that is, type 0, omitting type 3 grammars or equivalently 'all grammars above the finite-state level'). This is the sense in which the terms 'supra-regular grammar' or 'supra-regular processing' will be used for the rest of the paper.

(f) *Formal language theory and natural language*

As already suggested, Chomsky's primary interest in formalizing automata and organizing in this way was to provide an initial framework for understanding human natural language ('natural' meaning languages such as Warlpiri, French or English, as opposed to artificial languages such as mathematics, PROLOG or C++). In particular, Chomsky pointed out that English cannot be captured by a regular or 'finite-state' grammar (FSG) because it includes structures (particularly, phrase structures with multiple long-distance dependencies and recursive sentence structures) that are beyond any FSA's capabilities. He further argued that various linguistic phenomena of movement and sentence transformation (e.g. from active to passive) are beyond the capability of context-free languages as well, and thus that natural languages must occupy some broader subset of the type 0 grammars (which he termed 'transformational'). However, it quickly became clear that transformational grammars are in fact too powerful, requiring a cumbersome set of constraints to make grammars with this degree of power tractable. Subsequently, theoretical linguists working within this formal paradigm have gradually honed in on the level of computational power required for natural language [25,26]. After some years of suspicion that context-free grammars were up to the task, it is now clear that certain phenomena of natural language require context-sensitive grammars, and most researchers in this field now agree that human languages require 'mildly context-sensitive' grammars (MSCGs): grammars whose power is just a bit beyond those captureable by a context-free grammar [27–29].

What does it mean to say that 'natural languages require grammars at the mildly context-sensitive' level? First, note that any of these abstract classes of automata, including the weaker class of FSAs, contain many automata far beyond that of any human being. For instance, the Manhattan phone book is a finite list, easily captureable in a simple FSA that has one state for each name/number pair, but this language is far beyond the capacity of any human. Thus, the statement that 'human languages require grammars of at least the power of a finite state automaton' does not imply that the human brain could instantiate *any* FSA. Similarly, just because some animal species can be shown to do various tasks at the finite-state level, we cannot assume that they can induce *any* FSG. Whatever class of computational systems natural language entails, it will always be some subset of the categories of automata described in FLT (figure 2). Thus, in applying the theory to real organisms, we can use it as a general road map, but we never

expect any of these very abstract classes of grammars or languages to be co-extensive with our own capabilities (or those of animals or children).

This approach might correctly be termed 'syntactocentric' [30], but exploiting FLT in empirical work in no way denies the central importance of meaning in language. Rather, it reflects an analytical, 'divide and conquer' strategy that chooses one component of the vast complex of human language, focusing on form rather than content. Fortunately, this form-based approach has been immensely productive in computer science, underlying many of the technological advances we take for granted today, and thus does not seem too limited to be of interest. More importantly, the current understanding of the most complex signalling systems in other animals (for example, bird or whale 'song'), along with other rule-governed systems of humans (e.g. music) suggests that they, like formal languages, are focused on structure and *not* complex meaning encoded into units of the signal. The empirical approach we advocate here relies on explicit formal theories as the basis for experimental design, eschewing questions of signal meaning for the time being. Thus, 'language' in the formal sense used for the remainder of this paper means simply a set of strings defined by some grammar. No notion of meaning is entailed or implied for the 'grammars' we consider: they simply accept or reject strings as belonging to some language.

There are explicit theories that concern the information in signals [31] (although this is quite different from meaning, as emphasized by Shannon [32]), along with semantic models treating meaning in its own right [33,34]. There are also empirical paradigms that focus on the acquisition of meaning by children [35–37] and animals [38]. Finally, it is possible to combine artificial grammar learning with meaning in the laboratory to create 'artificial language learning' experiments [39]. There is thus no conflict between a focus on form (syntax or 'grammar') and content (meaning or 'language')—these are complementary fields of study that must, ultimately, be synergistically combined.

Given this caveat about the purely syntactic nature of FLT, one might ask why anyone should be interested in the question of where human (or animal) capabilities lie in the classification system of FLT. Here are a few reasons, ranging from practical to theoretical. From a purely practical viewpoint, scientists attempting to create computer programs that deal with corpora of data are greatly aided by knowing where the signal-generating system they are studying lies in this system. For instance, the parsing and compiling of either regular or context-free languages are well-defined problems with practical working solutions, but this is not true for context-sensitive languages (*sensu strictu*). The difference between a finite-state and context-free representation is also important to keep in mind when compiling computer code or analysing natural language texts.⁵ Knowing whether such texts demand context-free (or higher) powers, or not, is thus very valuable for anyone interested in building fast, robust computing systems. Similarly, recent years have seen an explosion of interest in complex animal signals such as bird song and whale song, and a vast amount of data have been

pouring in from both laboratories and the field that needs to be processed by computer [40]. The tools applied to this task need to be capable, at the formal level, of dealing with the actual complexity of these signals, and if such signals could be shown to require supra-regular grammars, the current crop of finite-state tools typically used to analyse them would be demonstrably inadequate.

In addition to these practical issues, there is a deeper theoretical reason for interest in the formal power of signal-generating systems. The core difference between human natural language and signal-generating systems such as music or birdsong, which can also generate highly complex signals, is that linguistic signals are used to transmit equally complex thoughts from one mind to another. Although music certainly communicates (e.g. mood or emotion, energy and many other powerful and subtle ‘messages’), there is no direct correspondence between the units of music (notes, phrases, etc.) and the structure of thought. For all its power and greatness, music simply cannot communicate plebian facts such as ‘the lion is in the third cave from the right’ or ‘you need to soak that nut in water and ashes for two days before you eat it’, nor can you use a musical phrase to represent those thoughts to yourself. Indeed, if (as in a few isolated cases) we use musical means such as drums or whistles to communicate thoughts, it ceases to be music and becomes language (‘drum talk’ or ‘whistle languages’). Thus, an intimate correspondence between signal and meaning is the *raison d’être* of language, and the key factor differentiating it from music, and, as far as we know, the diverse signalling systems of every other species on our planet.

There are many grounds for suggesting that thought itself has a tree-like, hierarchical structure [41]. Research in memory, category formation, word learning, visual cognition, Theory of Mind and many other fields all point to this conclusion [42–44], and Herbert Simon has advanced strong theoretical arguments for why a system of thought *must* have such a structure [45]. But if thoughts have a tree-like structure, and are unlimited in number, any signalling system capable of encoding thoughts (that is, any ‘language’ worthy of the name, possessing semantics and thus beyond the rigorous confines of FLT) must be able to capture this structure. This is a potentially deep reason that natural languages have, and arguably must have, hierarchical phrase structure (that is, must go beyond simply stringing items together in a simple finite-state system)—they would be inadequate vehicles for thought if they did not. Although, by introducing meaning, this argument clearly goes beyond FLT, it provides the broader context in which these questions become centrally important to anyone interested in natural language as a whole. This has been clear since the beginnings of the discipline: the so-called ‘weak generative capacity’ (the ability to match stringsets alone) is of quite limited interest. Ultimately, the ability to recover the phrase structure(s) underlying a string (roughly speaking, ‘strong generative capacity’) is much more interesting, and obviously critical for recovering structured thoughts from linear signal strings. If it could be shown that the signal-generating system of some particular species is limited to a simple serial

FSG (e.g. the chickadee calls of Hailman & Ficken [15] and Hailman *et al.* [46]), we need wonder no further why that system is not used to express unlimited combinatoric meanings and complex thoughts. Thus, although FLT only gives us tools for exploring signalling systems as stringset, not as meaningful systems, discovering whether the signals generated and processed in animal communication systems are limited to simple finite-state systems or not will have important ramifications for understanding their capacity to convey meaning, and ultimately for understanding the biology and evolution of language.

3. THE ROLE OF INFINITY IN FORMAL LANGUAGE THEORY

Before we turn to experimental work grounded in FLT, we will briefly discuss the controversial issue of ‘infinity’ in discussions of language. This is an old issue, nicely encapsulated in Wilhelm von Humboldt’s suggestions that human language makes ‘infinite use of finite means’ [47]. Clearly, every individual human has a finite memory, a finite lifespan and a finite (though astronomical) number of neurons and synapses. This has always been accepted [3,6,48]. Nonetheless, most linguists or computer scientists happily accept that any natural language such as English or Chinese is infinite (in the sense of ‘unbounded’), in precisely the same way that the set of the integers is infinite. One argument for this parallels the argument for numbers: if someone claims to have identified the largest possible integer, you can easily prove them wrong by simply adding one to their proposed number. In the same way, any proffered ‘longest sentence’ *x* can be trumped by simply generating ‘John thinks that *x*’. Although each of these sentences is of a fixed and a finite length (there are no infinite sentences), the set of sentences is infinite. From this mathematical perspective, we should no more doubt the infinity of English than we doubt the infinity of the integers.

However, there are more subtle arguments for and against the importance of infinity in natural language [49]. For example, the list of sentences a child hears before fixing on a grammar of English is surely finite, as is the list of all sentences an individual will produce in his/her lifetime. In principle, such lists could be captured by a finite-state system, in the extreme case simply as a list of those sentences. In contrast, all of the proofs used in FLT to demonstrate supra-regularity use the argument of infinity to prove their case (typically this involves invoking the pumping lemma [17]). From a strictly mathematical viewpoint, suspending the axiom that languages are infinite would invalidate most such proofs, and thus greatly weaken FLT.

However, no one supposes that the child simply memorizes all heard sentences: any language user can generate and understand novel sentences, beyond the finite input they received in childhood. Some system of more general or abstract rules is necessary to account for this ability. As Chomsky notes [3], ‘a grammar must reflect and explain the ability of a speaker to produce and understand new sentences which may be much longer than any he has previously heard’. The minimum that we might

need to account for such generalization is a probabilistic model over a finite-state system (some form or another of a 'Markov process'). But as Chomsky further observes in the same passage 'the point is that there are processes of sentence formation that this elementary model for language is intrinsically incapable of handling', and those include sentences with multiple embedding and nested- or crossed-dependency. Thus, 'the assumption that languages are infinite is made for the purpose of simplifying the description' [3], and to allow mathematical proofs that apply to all and every sentence. But there is no theoretical difficulty at all in limiting stack depth or tape length in supra-regular grammars.

Another convincing counter-argument, owing to Levelt [6], goes as follows. Let us assume that human language use *could* be modelled by a FSA, augmented with transition probabilities (a form of Markov process). For this model to have any psychological validity, a child would need enough data to infer these probabilities from the input. So we can ask what order of Markov approximation would be needed for typical sentences. This reduces to the question 'how many words can separate two words that are dependent upon one another in a sentence?' In the grammatical English sentence 'The woman you recently invited to come to New York and give a lecture in our department seems to be sick', 15 words intervene between the inter-dependent words 'woman' and 'seems'. An attentive English speaker will certainly notice if this pair were incorrectly inflected (e.g. 'The woman ... seem' or 'The women ... seems'). Hence, we would need a k -limited Markov source with $k = 15$ to capture this dependency reliably. But Levelt shows that even with unrealistically lenient assumptions [6, vol. 3, p. 76], such a Markov grammar would require an enormous number of parameters, on the order of 4^{15} , or more than one billion. The busy child would need to set about 30 parameters per second, throughout all of childhood, to assimilate such a model. Thus, a finite-state model must be limited to be learnable (and thus unable to deal with long-distance dependencies) *or* it could be theoretically adequate but, owing to the huge number of parameters, practically useless as a model of the child. This argument can be made from many different perspectives, but will always come to the same conclusion: given realistic assumptions, no regular grammar can adequately model English or any other natural language. This is both a practical and a theoretical conclusion.

This combination of arguments has led most commentators to accept the supra-regular hypothesis for humans. Of course, *any* model of human cognition will make simplifications, and thus will be inadequate in certain ways. This is intrinsic to model-building. What we seek are models that make the right generalizations, and that fail in reasonable ways. For example, we can make a simple modification of the weakest supra-regular system, a push-down automaton, in which the stack memory is of a fixed, limited depth. Such a model has no problem with long-distance dependencies, but it *will* have problems with multiple levels of embedding. This is precisely what is observed in humans experimentally, in abundant psycholinguistic research [50,51]. In other words, supra-regular models

with finite stores (limited stack depth or tape length) fail in ways that seem much more realistic as models of human performance.

In conclusion, we should not conclude from the importance of infinity in formal mathematical proofs that infinity plays a central role when we turn to practical empirical issues. Infinity is a powerful tool for abstraction, and its judicious use in mathematics allows a kind of certainty that is wonderfully satisfying. For example, Fourier's Theorem *proves* that any complex signal can be built up by a series of sine and cosine waves. Unfortunately, the proof requires an infinitely periodic signal (which continues unvarying from the infinite past to the infinite future), as well as an infinite set of sine waves. Despite these unrealistic assumptions, the discrete Fourier transform, applied to real signals, turns out to be an incredibly powerful tool at the heart of every mobile telephone and spectrographic programme on the planet today. In the same way, we can readily assume that PDA's stack or our Turing machine's tape will be of limited depth, and try to match this to empirical observations of finite humans. Although in doing so we lose the ethereal certainty of theorems, we lose few if any of the practically relevant insights of FLT.

4. EMPIRICAL INVESTIGATIONS USING FORMAL LANGUAGE THEORY

In the rest of this paper, we explore how FLT can be used, practically, by biologists, psychologists and neuroscientists, to design and execute experiments and analyse the resulting data. Often, such studies use artificial grammar learning (AGL) paradigms, see [52]. Because most of the recent literature reviving the supra-regular hypothesis has focused on the finite-state/context-free distinction, we start with a detailed investigation of one particularly simple supra-regular grammar: the 'counting grammar' A^nB^n , which has been the focus of numerous recent studies.

(a) A^nB^n : a model supra-regular grammar

The stringset defined by A^nB^n , in which the number of 'A' units is precisely matched by the number of 'B' units, has played a prominent role in the development of FLT. It is a textbook example of a simple language that cannot be captured by a regular grammar, as already discussed. Despite its ubiquity in the theoretical literature, to our knowledge, the first use of this grammar in experiments was that by Fitch & Hauser [8], who compared the acquisition of two different grammars in two different species: humans and cotton-top tamarins, a New World monkey species. One grammar was the simple regular grammar $(AB)^n$, which entails any number of 'AB' units, and the other was A^nB^n . In both cases, the units were consonant-vowel speech syllables, with the A units spoken by a human female and the B units by a male. Fitch & Hauser found that, while college undergraduates were able to master both grammars, the monkeys only showed above-chance rejection of non-grammatical stimuli for the regular grammar. The monkey's success on the regular grammar showed that the techniques were adequate to